CSCI 2406 | Computer Organization & Architecture | TOPIC 10

# Single-Cycle Microarchitecture & Datapath

AArch64 datapath components, control signals, and a worked `ldr` instruction path

Dr. J. Burns

# Topic 10 Outline

1. **Single-Cycle Microarchitecture**
   - Architectural state and execution within one clock period.
2. **State Elements and Core Datapath Blocks**
   - PC, memories, register file, ALU, and immediate generator.
3. **Worked Load Path**
   - The complete path for `ldr x7, [x5, #24]`.
4. **Extending the Datapath**
   - PC-relative branches and the complete extended datapath.
5. **Control Signals and Timing**
   - Control settings and the single-cycle clock-period limit.
6. **Summary and Review**
   - Summary and self-test questions.

Section 1

# Single-Cycle Microarchitecture

# Microarchitecture

- The **ISA** defines the instructions visible to software.
- The **microarchitecture** is the hardware organization that executes those instructions.
- This single-cycle datapath implements a small subset of AArch64 instructions.
- The datapath moves values through memories, registers, ALUs, and multiplexers.
- The control unit interprets the instruction and selects the datapath actions.
- Later implementations can use pipelining while preserving the same ISA behavior.

# State Elements

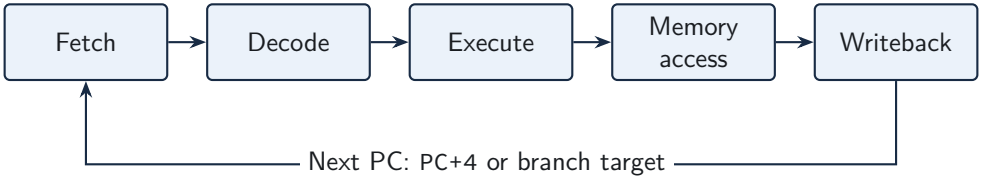
**State** is information retained between clock cycles. A **state element** is hardware that stores that information.

- **Program counter (PC)** stores the address of the current instruction.
- **Register file** is the hardware storage block holding general-purpose register values, with ports for reading and writing selected registers.
- **Memory** stores program instructions and data. The diagrams show separate instruction-fetch and load/store interfaces to this memory.
- **Combinational logic** computes outputs from its inputs; it does not retain state between cycles.

In this model, enabled writes take effect at the closing clock edge.

# Single-Cycle Execution

One instruction completes in one clock cycle.




- These five activities occur within one cycle, with no intermediate stage registers.
- At the closing clock edge, the PC receives the next instruction address and other enabled state updates take effect.
- The return arrow shows selection of the next instruction, not a data connection from writeback to the PC.

# Single-Cycle Execution: Activities

One instruction completes in one clock cycle. Its required work must settle before the closing clock edge.

| Activity | Meaning |
|---|---|
| Fetch | Read the 32-bit instruction at the PC address. |
| Decode | Identify its operation and fields; select register operands and control settings. |
| Execute | Perform the ALU operation or calculate an address or branch target. |
| Memory access | Read data for a load or write data for a store. |
| Writeback | Write a result to a destination register, when required. |

These are activities within one cycle, not separate clocked stages. Not every instruction needs each activity. The PC and enabled state updates occur at the closing edge.

# AArch64 Architectural State

Architectural state is the information defined by the ISA that determines program execution.

| State | Purpose |
|---|---|
| PC | Current instruction address; separate from the integer registers. |
| `x0`–`x30` | 31 general-purpose integer registers, each holding 64 bits. |
| `sp` | Stack pointer; omitted from this simplified datapath. |
| NZCV | Condition flags; storage and condition-check logic are omitted. |
| Memory contents | Program instructions and data bytes. |

- Register specifiers are 5-bit instruction fields, not stored register values.
- Encoding 31 denotes `sp` or `xzr`, depending on the instruction form. `xzr` reads as zero and discards writes; it holds no mutable state.

# Instructions Covered

These are the instructions supported by the single-cycle datapath developed throughout this deck.

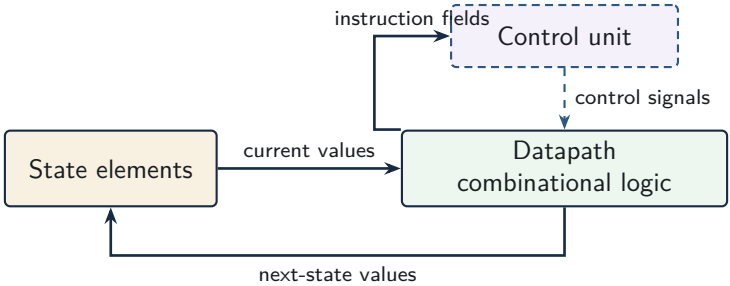
- **Memory**: `ldr xT, [xN, #imm]` and `str xT, [xN, #imm]` using unsigned scaled offsets.
- **Data processing**: unshifted register forms of add, sub, and, and `orr`, plus unshifted immediate add/sub.
- **Control flow**: unconditional `b label`.
- The load instruction is traced first because it exercises fetch, register read, immediate generation, ALU address calculation, data memory, and writeback.

Examples use `x0`–`x30`. Handling `sp`/`xzr` requires register-31 logic; conditional branches require flag storage and condition-check logic. These additions are not shown.

Section 2

# State Elements and Core Datapath Blocks

# Datapath and Control



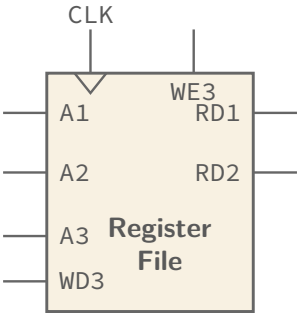

At the clock edge, enabled state elements capture the next-state values.

- Control enables register or memory writes and selects operands, ALU operations, immediate generation, writeback data, and the next PC.

# Register File

The **register file** stores the general-purpose register values and provides access ports. The ISA defines the registers; this datapath implements two read ports and one write port.

- `A1` and `A2`: 5-bit register numbers selecting the two reads.
- `RD1` and `RD2`: the two 64-bit register values.
- `A3`: 5-bit destination register number.
- `WD3`: the 64-bit value to write.
- `WE3`: write enable; the selected register changes at the clock edge.

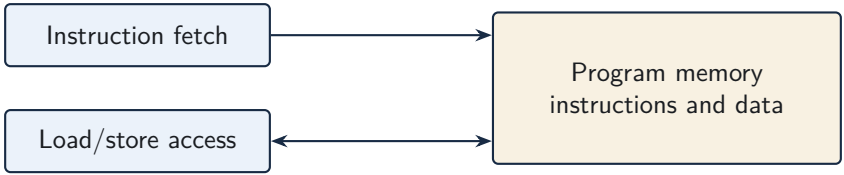



Two reads supply both operands of add `x1, x2, x3`. Reads are combinational; enabled writes occur at the clock edge.

# Instruction Memory and Data Memory

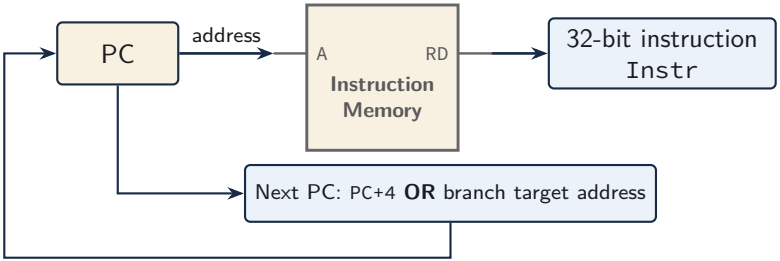
The program's memory address space contains both instruction bytes and data bytes.

- **Instruction memory**: the access interface used to fetch an instruction at the PC address.
- **Data memory**: the access interface used by `ldr` and `str` to read or write data at an effective address.
- The diagrams separate these roles logically. They do not imply two separate process memories.




A single-cycle load needs instruction fetch and data access within the same cycle. Hardware must provide both accesses, for example through separate ports or instruction/data caches.

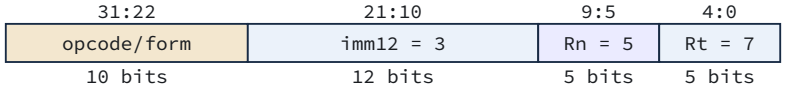
# Program Counter and Instruction Memory

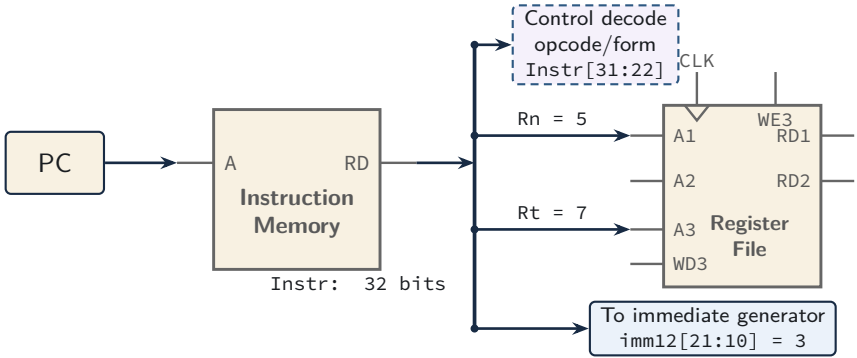



- The instruction-memory interface fetches instruction bytes from program memory: A receives the PC; RD returns the 32-bit word.
- Instruction memory is read combinationally in this single-cycle datapath.
- Sequential execution uses PC+4; b selects a branch target. The PC captures its next value at the clock edge.

# Parsing the Instruction Word

Example: `ldr x7, [x5, #24]`.



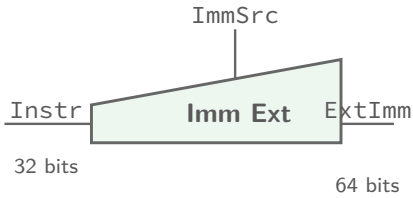




For this load: `Rn` → `A1`, `Rt` → `A3`; `A2` is unused.

Bits `[4:0]` are `Rt` for `ldr/str`, and `Rd` for arithmetic instructions.

# Immediate Generator

- `Instr` supplies the instruction bits.
- `ImmSrc` selects the field and the required extension/scaling mode.
- The selected value is zero-extended or sign-extended to 64 bits.
- Load/store and branch offsets are also scaled when required.
- `ExtImm` is the resulting 64-bit datapath value.




This is combinational logic: it has no clock input and stores no state.

# AArch64 Immediate Modes in This Datapath

`ImmSrc` selects the generation rule after the instruction form has been decoded.

| **Instruction form and example** | **Encoded immediate** | **Generated 64-bit value** |
|---|---|---|
| Unshifted add/sub immediate<br>`add x1, x2, #24` | `imm12[21:10]`<br>Field value: 24 | Zero-extend `imm12`; no scaling.<br>Result: 24. |
| Unsigned-offset `ldr/str x`<br>`ldr x7, [x5, #24]` | `imm12[21:10]`<br>Field value: 3 | Zero-extend `imm12`, then multiply by 8.<br>Result: 24-byte offset. |
| Unconditional branch<br>`b label`<br>Target 8 bytes earlier | `imm26[25:0]`<br>Signed value: $-2$ | Append two zero bits, then sign-extend:<br>`SignExtend(imm26:00, 64)`.<br>Result: $-8$-byte offset. |

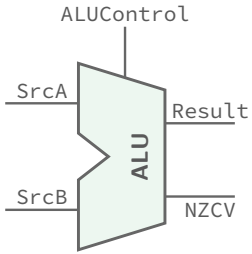
Load/store scaling is 8 here because the supported transfers are 64-bit. Branch offsets are added to the address of the branch instruction itself.

# Arithmetic Logic Unit (ALU)

- `SrcA` and `SrcB`: two 64-bit operands.
- `ALUControl`: a 2-bit operation selector in this design.
- `Result`: the 64-bit arithmetic/logical result or effective address.

| ALUControl | Operation |
|---|---|
| `00` | Add: `SrcA + SrcB` |
| `01` | Subtract: `SrcA - SrcB` |
| `10` | Bitwise AND |
| `11` | Bitwise OR |

These example control codes are chosen for this datapath; they are not AArch64 instruction encodings.

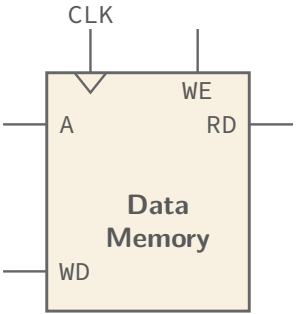



The ALU is combinational. NZCV flags require separate storage and a flag-write enable; these are omitted here.

# Data Memory

The data-memory interface accesses the program's data bytes for loads and stores.

- `A`: the byte address computed by the ALU.
- `RD`: the 64-bit value read for `ldr xT`.
- `WD`: the 64-bit register value supplied by `str xT`.
- `WE = MemWrite`: enables a memory write at the clock edge.
- Reads are combinational in this single-cycle datapath; instruction and data memory are separate logical interfaces.




Fixed read delays are assumed; caches, exceptions, and memory-mapped I/O are omitted.

Section 3

# Worked Load Path

# Load Example

`ldr x7, [x5, #24]`

- Read the base address from `x5`.
- Add the byte offset `24` to form the effective address.
- Read 8 bytes from data memory.
- Write the loaded 64-bit value into `x7`.
- Update the PC to the next sequential instruction at the end of the cycle.

For a 64-bit AArch64 load with unsigned offset, the encoded `imm12` is $24/8 = 3$.

# Load Instruction Fields

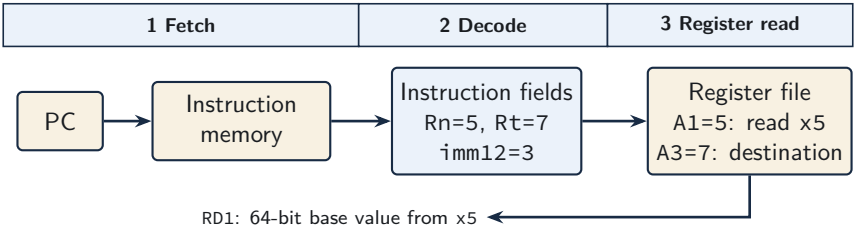
`ldr x7, [x5, #24]` is encoded as `0xF9400CA7`.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Binary | 1111 | 1001 | 0100 | 0000 | 0000 | 1100 | 1010 | 0111 |
| Hex | **F** | **9** | **4** | **0** | **0** | **C** | **A** | **7** |

| Field | Value | Meaning |
|---|---|---|
| opcode/form | `1111100101` | 64-bit load, unsigned immediate |
| `imm12` | `000000000011` | scaled offset 3, meaning 24 bytes |
| `Rn` | `00101` | base register `x5` |
| `Rt` | `00111` | destination register `x7` |

# Steps 1–3: Fetch, Decode, and Read the Base Register

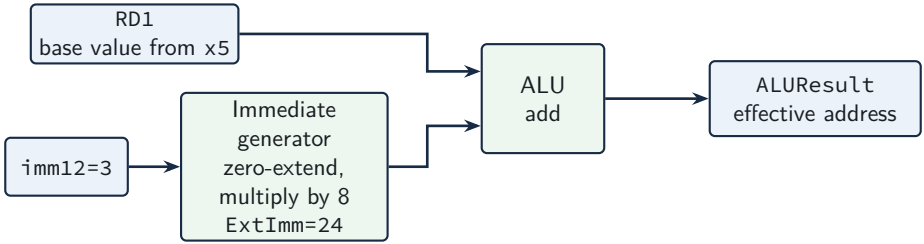
```
ldr x7, [x5, #24]
```




1. **Fetch:** the PC addresses instruction memory, which returns instruction word `0xF9400CA7`.
2. **Decode:** identify the load and its fields. `Rn` selects the base register; `Rt` selects the destination.
3. **Read:** the register file supplies the contents of `x5`. Immediate generation can proceed alongside this read; address calculation needs both results.

# Steps 4–5: Generate the Offset and Compute the Address

```
ldr x7, [x5, #24]
```



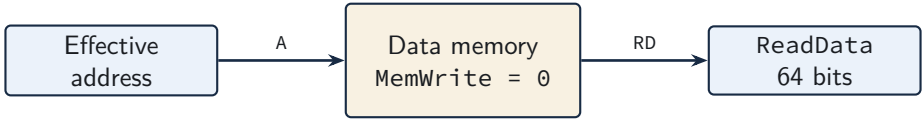

4. **Generate the byte offset.** This is an 8-byte load, so the encoded `imm12=3` becomes `ExtImm=24`.

5. **Calculate the effective address.** The ALU adds the base value read from `x5` to `ExtImm`:

   `Effective address = contents of x5 + 24.`

# Step 6: Read Data Memory

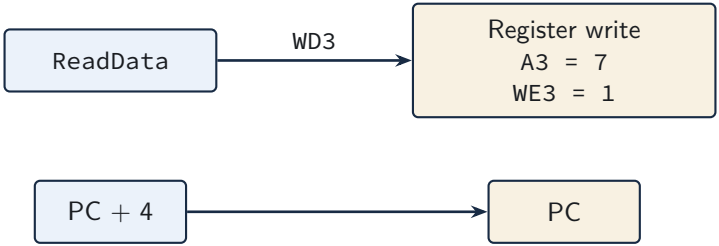
```
ldr x7, [x5, #24]
```




The memory read value becomes the candidate writeback value for `x7`.

# Step 7: Write Back and Update PC

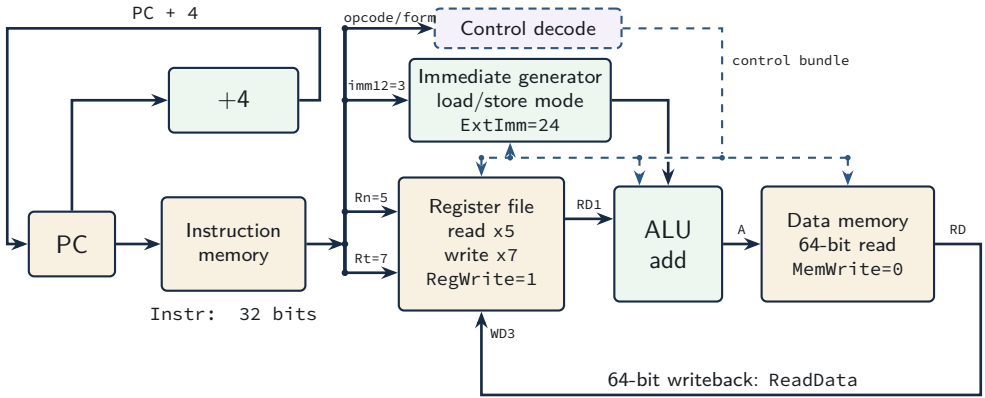
```
ldr x7, [x5, #24]
```




At the clock edge, x7 receives the loaded value and the PC receives $PC + 4$.

# Complete Load Datapath

```
ldr x7, [x5, #24]
```




At the clock edge, x7 receives the 64-bit read value and the PC receives $PC + 4$. Dashed outlines identify control logic; dashed bundles carry separate control signals.

# Load Path Control Settings

| Signal | Setting for `ldr x7, [x5, #24]` | Effect |
|---|---|---|
| `RegWrite` | 1 | write loaded value to `x7` |
| `MemWrite` | 0 | do not write data memory |
| `MemToReg` | 1 | writeback comes from data memory |
| `ALUSrc` | 1 | ALU second input is the extended offset |
| `ALUControl` | add | compute base + offset |
| `ImmSrc` | load/store scaled | produce byte offset 24 |
| `PCSrc` | 0 | next PC is $PC + 4$ |

These control values select one path through the same hardware.

Section 4

# Extending the Datapath

# Branching in AArch64: Forward and Backward

A branch changes which instruction is fetched next. `b label` transfers control unconditionally to `label`.

## The branch instruction supplies the base address

$$\text{Target} = PC_{\text{branch}} + \text{signed byte offset}$$

$PC_{\text{branch}}$ is the address of the b instruction itself.

**Forward: skip an instruction**

```
       b ahead       // 0x1000
       add x0,x0,#1  // 0x1004
ahead: sub x1,x1,#1  // 0x1008
```

`0x1008 - 0x1000 = +8 bytes`

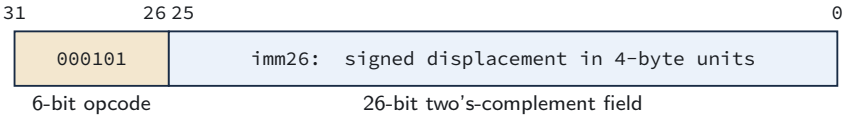
**Backward: repeat instructions**

```
again: add x0,x0,#1   // 0x2000
       sub x1,x1,#1   // 0x2004
       b again        // 0x2008
```

`0x2000 - 0x2008 = -8 bytes`

The assembler calculates the displacement from the label. Each A64 instruction occupies 4 bytes.

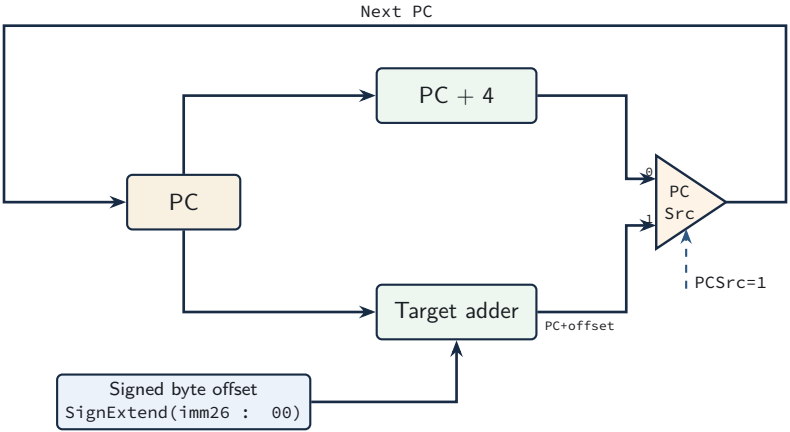
# B Instruction: Encoding the Offset



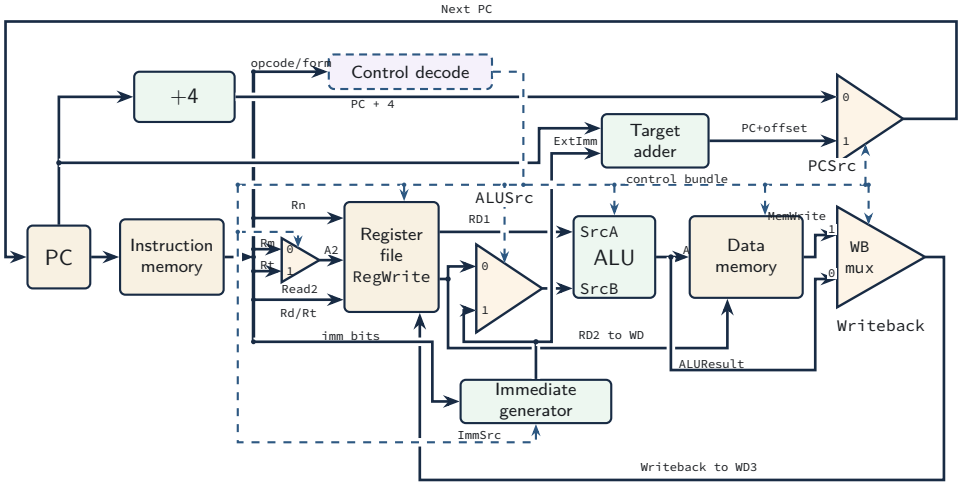

- The label is represented by a displacement, not an absolute address.
- `imm26 = (Target - PC_branch) / 4`.
- Append two zero bits, then sign-extend to 64 bits:
  `offset = SignExtend(imm26 : 00)`. Here `:` joins bit fields.

| Byte offset | Signed `imm26` | Encoded field | 32-bit instruction |
|---|---|---|---|
| +8 | +2 | 0x0000002 | 0x14000002 |
| −8 | −2 | 0x3FFFFFE | 0x17FFFFFE |

# Branch Path: `b label`



# Complete Extended Datapath




Dashed bundles: separate control signals. Triangles: muxes. Trace add and `str` in Self-Test 2.

Section 5

# Control Signals and Timing

# How Control Selects an Instruction Path

- The datapath contains several possible routes. The control decoder recognizes the instruction form and selects the route needed for that instruction.
- **Write enables** determine which state can change: `RegWrite` for a register and `MemWrite` for data memory.
- **Multiplexer selectors** choose operands, writeback data, and the next PC.
- **Operation controls** choose the ALU operation and immediate-generation mode.

**Example: `add x1, x2, x3`**

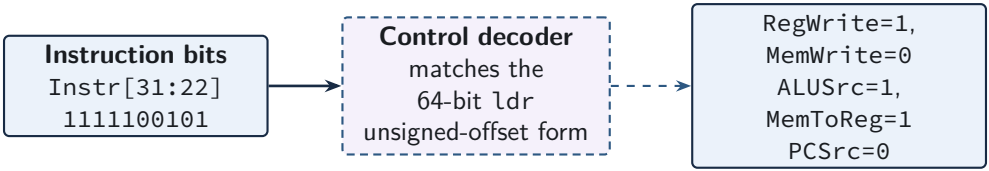
Read `x2` and `x3`; select the register operand and ALU addition; write the ALU result to `x1`. Do not write data memory. Select `PC+4`.

# Control Signals

| Signal | Meaning |
|---|---|
| `RegWrite` | enable a write to the register file |
| `MemWrite` | enable a write to data memory |
| `MemToReg` | choose memory data or ALU result for writeback |
| `ALUSrc` | choose register data or immediate as ALU input B |
| `ALUControl` | select add, subtract, and, or other ALU operation |
| `ImmSrc` | select the immediate field, extension, and scaling |
| `Read2Src` | choose `Rm` or `Rt` for the second register read |
| `PCSrc` | choose sequential PC or branch target |

# Instruction Decoding and Control Signals

For `ldr x7, [x5, #24]`, the instruction word is `0xF9400CA7`. The decoder recognizes the supported instruction form and generates its control settings.



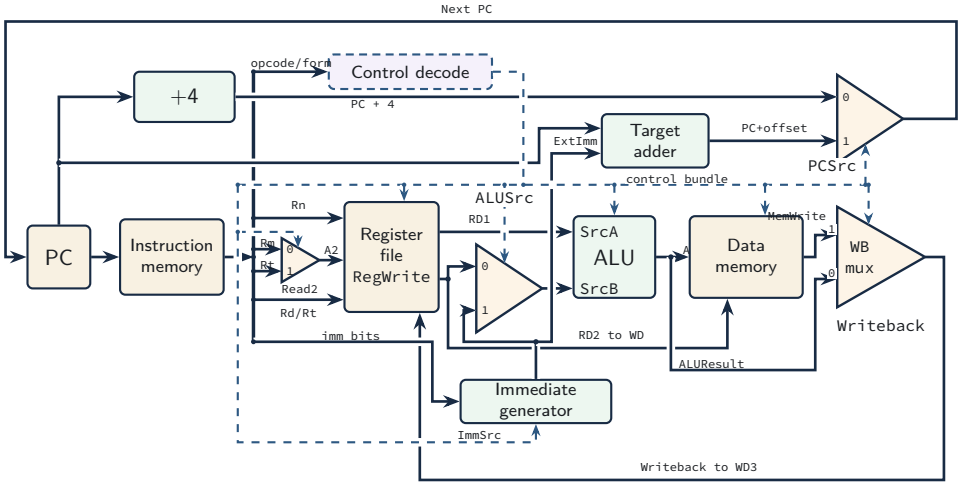

- These ten bits identify the access size, instruction form, and load operation; they are not a single indivisible opcode field.
- The decoder also selects `ALUControl=add` and the load/store immediate mode: zero-extend `imm12` and multiply by 8.
- Register fields select `x5` and `x7`; `imm12=3` produces byte offset 24. The operand fields select registers and offset; the instruction form determines the control settings.

# Control Examples: Load, Store, and Branch

- **`ldr x7, [x5, #24]`**: `ALUSrc=1`, `ALUControl=add`, and `ImmSrc=load/store`. Select memory for writeback (`MemToReg=1`); enable the register write (`RegWrite=1`). Keep `MemWrite=0`.
- **`str x7, [x5, #24]`**: use the same address calculation. Select `Rt=x7` for read port 2 (`Read2Src=1`) and send its value to memory. Set `MemWrite=1`, `RegWrite=0`.
- **`b label`**: select the signed, scaled branch displacement and `PCSrc=1`. The target adder supplies the next PC. Keep both write enables at 0.

For the load and store, `PCSrc=0` selects `PC+4`. All settings settle during the cycle; enabled state changes occur at its closing edge.

# Complete Extended Datapath




Dashed bundles: separate control signals. Triangles: muxes. Trace add and `str` in Self-Test 2.

# Example Control Settings

The settings below match our datapath diagram; other processor designs may use different signals or encodings.

| Instruction | RegW | MemW | MemToReg | ALUSrc | ALU | ImmSrc | Read2Src | PCSrc |
|---|---|---|---|---|---|---|---|---|
| `ldr xT, [xN, #imm]` | 1 | 0 | 1 | 1 | add | scaled offset | X | 0 |
| `str xT, [xN, #imm]` | 0 | 1 | X | 1 | add | scaled offset | 1 | 0 |
| `add xD, xN, xM` | 1 | 0 | 0 | 0 | add | X | 0 | 0 |
| `sub xD, xN, xM` | 1 | 0 | 0 | 0 | sub | X | 0 | 0 |
| `add xD, xN, #imm` | 1 | 0 | 0 | 1 | add | arithmetic imm | X | 0 |
| `b label` | 0 | 0 | X | X | X | branch offset | X | 1 |

- **Immediate modes:** scaled offset = zero-extend `imm12`, multiply by 8; arithmetic immediate = zero-extend `imm12`; branch offset = sign-extend `imm26:00`.
- `Read2Src`: 0 selects `Rm`; 1 selects `Rt`. `X`: either setting gives the same architectural result.
- **Scope:** 64-bit operands; unsigned-offset loads/stores; unshifted arithmetic; register-31 handling omitted. This table shows representative instructions.

# Single-Cycle Clock Period

The clock period must accommodate the longest state-to-state path. In this datapath, a load typically determines that limit.

- The loaded value must pass through address calculation, data memory, and the writeback multiplexer, reaching the register-file write input before the closing edge.
- Allow for source-register clock-to-output delay, combinational propagation delays, and destination-register setup time.
- Register reading and immediate generation can overlap. Their delays are not simply added together: the ALU needs both its operands and control settings to settle.
- Check every relevant data and control path, including the next-PC path. The longest required path sets the minimum clock period.
- Shorter instructions such as add and b still use that same period, even when their results are ready earlier.

# Single-Cycle Limitations

- **Fixed clock period:** every instruction occupies the full period required by the longest path, typically a load.
- **Uneven hardware use:** add bypasses data memory, while b uses the target adder. Their shorter paths do not reduce instruction time.
- **Memory access capacity:** a load needs both instruction fetch and data access within one cycle. A memory permitting only one access per cycle cannot serve both.
- **Arithmetic resources:** sequential-PC calculation and operand/address calculation need separate resources when required together. This datapath also provides a branch-target adder.
- **Memory delay:** the model assumes bounded read latency. Cache misses and variable-latency memory require additional mechanisms, such as wait cycles, outside the strict one-instruction-per-cycle model.

# Looking Ahead

- The single-cycle datapath shows the full instruction path in one cycle.
- Pipelining splits that path into stages so several instructions can be in progress at once.
- The next topic uses the same fetch, decode, execute, memory, and writeback structure to build the 5-stage RISC pipeline.
- Hazards appear when overlapped instructions need the same data or control decision.

Section 6

# Summary and Review

# Topic 10 Summary

- A single-cycle datapath computes the next architectural state in one clock period.
- The state elements shown store the PC, integer register values, and memory contents.
- Instruction fields select register ports, immediate-generation modes, and control settings.
- The `ldr x7, [x5, #24]` path uses instruction fetch, register read, immediate scaling, ALU address calculation, memory read, and register writeback.
- The same datapath is extended with multiplexers and control signals for stores, ALU instructions, and branches.
- `b` adds a signed PC-relative byte offset to the address of the branch instruction; `imm26` encodes that offset in 4-byte units.

# Self-Test 1: Datapath Basics

1. In a single-cycle datapath, when are state elements updated?
2. Why is the PC modeled separately from the AArch64 general-purpose register file?
3. For `ldr x7, [x5, #24]`, which fields select the base and destination registers? Which register-file inputs receive them?
4. Why is the encoded `imm12` equal to 3 rather than 24 for that instruction?
5. What is the role of the ALU in a load or store instruction?
6. What does the vertical instruction bar represent? Does it store state?

# Self-Test 2: Control and Timing

1. Identify the four triangular multiplexers in the complete extended datapath. What two data inputs and one output does each have? Which signal controls the writeback multiplexer?
2. Why does `str` assert `MemWrite` but not `RegWrite`?
3. Trace `add x1, x2, x3` and `str x7, [x5, #24]` through the complete diagram. For each, identify the register fields, ALU inputs, selector settings, memory action, register write (if any), and next PC.
4. Why does a single-cycle implementation have a long clock period?
5. Which datapath outputs contribute to the architectural result of `add x1, x2, x3` but not `b label`?
6. A `b` at `0x2008` targets `0x2000`. What are the byte offset, signed `imm26`, and next PC? Which address is the base?